

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	50% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	CHARANDEEP.N.C	Reg. No.:	192472196
Section / Batch:	CSE-AI	Date:	3-08-2026

Code of Conduct

I Charandeep/192472196 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *charandeep*

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Part of Speech Tagging	Morphological decomposition of connected, connecting, and connection; identification of roots, suffixes, inflectional vs. derivational forms, and normalization for document indexing.	Q1
English Word Classes, Tagsets for English, Rule-Based Part of Speech Tagging	Morphological parsing of unhappy, happiness, and happily; identification of prefixes, suffixes, base forms, and semantic effects of derivational morphology.	Q2
Counting Words, Part of Speech Tagging, English Word Classes	Stemming and root extraction for played, player, and playing; handling grammatical variations and word formation changes for search preprocessing.	Q3
Finite-State Morphological Analysis, Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Finite-state parsing of writes, writing, and written; modeling regular and irregular verb inflections and root normalization.	Q4
English Word Classes, Rule-Based Part of Speech Tagging, Counting Words	Porter Stemmer-based processing of relational, relation, and relate; application of derivational suffix removal and stem extraction for retrieval.	Q5

Annexure A – ANALYTICAL PROBLEM SOLVING

1. Given the input words: “connected”, “connecting”, “connection”, design a morphological analysis pipeline for a document indexing system.

Apply rules to decompose each word into root and suffix components.

Differentiate between inflectional and derivational forms within the dataset.

Normalize all variants to a common base form for efficient retrieval.

Determine the parsed structure and normalized output for each word.

2. Given the input words: “unhappy”, “happiness”, “happily”, design a morphological parsing module for sentiment analysis.

Identify prefixes, suffixes, and base forms using rule-based decomposition.

Ensure semantic consistency across derived word forms.

Classify each transformation as inflectional or derivational.

Produce the root and morphological breakdown for each word.

3. Given the input words: “played”, “player”, “playing”, design a stemming-based preprocessing module for a search engine.

Apply morphological rules to strip affixes and identify the base form.

Handle both grammatical variations and word formation changes.

Ensure consistent root extraction across all forms.

Determine the stem and classification for each input word.

4. Given the input words: “writes”, “writing”, “written”, design a finite-state morphological parsing module for a text processing system.

Apply state transitions to represent suffix transformations and irregular forms in verb inflection.

Differentiate between regular and irregular inflectional patterns within the dataset.

Ensure consistent mapping of all forms to a common root representation.

Determine the state transitions, morphological breakdown, and normalized output for each word.

5. Given the input words: “relational”, “relation”, “relate”, design a Porter Stemmer-based preprocessing module for document retrieval.

Apply stemming rules step-by-step to remove derivational suffixes and obtain base forms.

Handle variations in suffix patterns while preserving semantic consistency.

Ensure uniform root extraction across all derived forms.

Determine the stemming steps, intermediate forms, and final stem for each input word.

Rubrics for Calculation

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q. No. / Criteria Level	Problem Understanding	Method Selection	Solution Process	Accuracy of Calculation	Interpretation of Results	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Problem 1 — Morphological Analysis Pipeline for Document Indexing

Input words: “connected”, “connecting”, “connection”

A document indexing system must recognize that many surface word forms refer to the same underlying concept, so that a search for one form retrieves documents containing any related form. This requires a pipeline that can decompose a word into its meaningful parts (morphemes), decide whether the transformation is purely grammatical or creates a new word, and then normalize accordingly.

Pipeline Stages

- Stage 1 – Tokenization: split the incoming text stream into individual word tokens and lower-case them for uniform processing.
- Stage 2 – Lexicon Lookup: check the token against a dictionary of known root forms; if an exact match exists, no further decomposition is needed.
- Stage 3 – Suffix Stripping: if no exact match is found, apply a rule set that strips inflectional suffixes (-ed, -ing, -s) before derivational suffixes (-ion, -tion, -ment) are considered, since inflectional morphology sits ‘outer’ to derivational morphology in English word structure.
- Stage 4 – Classification: tag each stripped suffix as Inflectional (same part of speech, purely grammatical variation such as tense or number) or Derivational (changes part of speech or creates a semantically distinct new lexeme).
- Stage 5 – Normalization and Indexing: collapse inflectional variants to the common verb root for the primary index key; retain derivational forms as linked but separately labeled entries so that both broad recall (via the root) and precise retrieval (via the exact derived form) are supported.

Design Approach

- Tokenize the input word and match it against a lexicon of known roots using longest-suffix-match rules.
- Apply a rule set that strips inflectional suffixes (-ed, -ing, -s) before derivational suffixes (-ion, -tion, -ment) are considered.
- Tag each stripped suffix as Inflectional (same part of speech, grammatical variation) or Derivational (changes part of speech / creates a new lexeme).
- Normalize inflectional variants to the common verb root; keep a separate but linked index entry for derivational nouns so exact-form retrieval is still possible.

Parsed Structure and Normalized Output

Word	Root	Suffix	Type	Rule Applied	Normalized Form
connected	connect	-ed	Inflectional	V + ed → past tense/participle	connect
connecting	connect	-ing	Inflectional	V + ing → present participle	connect
connection	connect	-ion	Derivational	V + ion → Noun (verb→noun shift)	connect (indexed also as ‘connection’ noun form)

Conclusion: “connected” and “connecting” are pure inflectional variants of “connect” and collapse directly to it in the index. “connection” is derivational — it changes the syntactic category (verb → noun) — so the indexing system links it to the same root cluster but preserves it as a distinct semantic entry for precision retrieval. In practice this means a search query for ‘connect’ would retrieve documents containing ‘connected’, ‘connecting’, and ‘connection’ alike, while a more specific query for ‘connection’ (e.g., in a networking context) could still be ranked with awareness that it is a nominal derivative rather than a verb form, improving both recall and downstream relevance ranking.

Problem 2 — Morphological Parsing Module for Sentiment Analysis

Input words: “unhappy”, “happiness”, “happily”

Sentiment analysis systems cannot simply strip affixes and normalize to a shared root the way a search engine does, because affixes such as negation prefixes carry sentiment-bearing information themselves. A morphological parser built for this purpose must therefore preserve the semantic contribution of every affix rather than discarding it, and must classify each transformation carefully so that opposite-polarity words are never merged into the same sentiment bucket.

Design Approach

- Detect prefixes first (e.g., un-, dis-, in-) since they typically reverse polarity — critical for sentiment scoring.
- Detect suffixes next (-ness, -ly, -ity) using a derivational suffix table mapped to the part-of-speech they create.
- Every transformation here changes either polarity or part of speech, so all three are classified as Derivational (none are grammatical/inflectional variants of ‘happy’).
- Maintain semantic consistency by keeping a polarity flag: ‘un-’ flips sentiment polarity so ‘unhappy’ must NOT be normalized to the same sentiment class as ‘happy’, even though it shares the root.

Root and Morphological Breakdown

Word	Prefix	Root	Suffix	Type	Semantic Effect
unhappy	un-	happy	—	Derivational	Polarity reversal (negative sentiment)
happiness	—	happy	-ness	Derivational	Adjective → Noun (state/quality)
happily	—	happy	-ly	Derivational	Adjective → Adverb (manner)

Conclusion: For sentiment analysis, the root “happy” anchors all three words for lexical lookup, but the prefix ‘un-’ must be handled as a polarity-inverting operator rather than simply stripped, so that “unhappy” is correctly scored as negative rather than being conflated with the positive root. “happiness” and “happily” retain the positive polarity of the root but shift its grammatical category (to noun and adverb respectively), so the sentiment module can still credit them toward a positive score while recording that they modify a noun phrase or a verb phrase rather than describing a person directly. This three-way treatment — polarity flag, category change, and shared semantic root — allows the module to remain both linguistically accurate and sentiment-aware.

Problem 3 — Stemming-Based Preprocessing Module for a Search Engine

Input words: “played”, “player”, “playing”

Stemming for search-engine preprocessing prioritizes recall over grammatical precision: the goal is to collapse every surface form that a user might type into a single index key, so a query using any variant retrieves all relevant documents. Unlike the sentiment-analysis case, the exact part of speech is not critical here — what matters is maximum overlap between query terms and indexed terms.

Design Approach

- Apply suffix-stripping rules (Porter-style) in order of suffix length: -ing, -ed, -er.
- Classify each stripped suffix: -ed and -ing are grammatical (inflectional) variations of the verb; -er is a word-formation (derivational) suffix that creates an agent noun.
- For search-engine recall, all three are mapped to the same stem ‘play’ so that a query for any form retrieves documents containing the others.

Stem and Classification

Word	Stripped Suffix	Stem	Classification	Rule
played	-ed	play	Inflectional	Remove past-tense suffix
player	-er	play	Derivational (agent noun)	Remove agentive suffix -er
playing	-ing	play	Inflectional	Remove progressive suffix

Conclusion: All three forms stem to “play”. This aggressive stemming favors recall (typical for search engines) — the module simply flags ‘player’ as derivational internally in case a more precision-oriented ranking stage needs to distinguish the agent-noun sense from the verb forms. For example, a query for ‘play’ would surface documents mentioning ‘played’, ‘playing’, and ‘player’ alike, maximizing recall; a later re-ranking stage can still down-weight ‘player’ matches if the query context indicates the user is looking for the action of playing rather than a person who plays.

Problem 4 — Finite-State Morphological Parsing Module

Input words: “writes”, “writing”, “written”

A finite-state approach models morphological transformation as a path through a small set of states connected by transitions labeled with letter operations (insertion, deletion, substitution). This works cleanly for regular verbs, whose suffixation follows predictable spelling rules, but English has a class of irregular (strong) verbs whose past and participle forms do not follow any general letter-transformation rule and must instead be captured by an exception table consulted before the regular transition network is applied.

Design Approach — Finite-State Model

- Define states: Q0 (root lexicon lookup) → Q1 (regular suffix attachment) → Q2 (accept state / normalized output).
- Regular forms follow the transition Q0 —write→ Q1 —(+s / +ing, with e-deletion before -ing)→ Q2.
- Irregular forms (e.g., ‘written’ as the past participle of ‘write’) bypass the regular suffix-transition path and are resolved via a direct irregular-verb lookup table edge straight to Q2, since they do not follow a predictable letter-for-letter transition.
- This distinguishes regular inflectional patterns (predictable FSA path) from irregular inflectional patterns (lookup-table shortcut), while still mapping every form to one normalized root.

State Transitions, Breakdown, and Normalized Output

Word	Root	Transformation	FSA Path	Pattern Type	Normalized Form
writes	write	write + -s	Q0→Q1(+s)→Q2	Regular inflectional	write
writing	write	write (drop ‘e’) + -ing	Q0→Q1(e-deletion,+ing)→Q2	Regular inflectional	write
written	write	suppletive/irregular past participle	Q0→Irregular-lookup→Q2	Irregular inflectional	write

Conclusion: “writes” and “writing” follow regular, rule-predictable FSA transitions, while “written” requires an irregular-verb lookup table because English strong verbs like ‘write’ use ablaut/suppletive participle forms rather than simple suffixation. All three normalize to the root “write.” This two-tier design (regular transition network plus irregular exception table checked first) is the standard

architecture used in real finite-state morphological analyzers, since it keeps the transition network small and general while still achieving full coverage of the language's irregular verb inventory.

Problem 5 — Porter Stemmer-Based Preprocessing Module for Document Retrieval

Input words: “relational”, “relation”, “relate”

The Porter Stemmer is a widely used rule-based algorithm that removes common English suffixes in a fixed sequence of numbered steps, each guarded by a condition on the ‘measure’ (m) of the stem — roughly, the number of consonant-vowel sequences remaining. Applying the steps in order, rather than all at once, prevents over-stemming and ensures that related derivational forms converge on the same final stem, which is exactly what a document retrieval index needs for consistent term matching.

Design Approach — Porter Stemmer Steps

- Step 1 (plurals/participles): check for -s, -ed, -ing endings — none apply directly here.
- Step 2 (derivational suffixes, condition $m > 0$): rule ‘ATIONAL → ATE’ converts relational → relate.
- Step 4 (further derivational suffixes, condition $m > 1$): rule removing ‘-ION’ when preceded by S or T converts relation → relat.
- Step 5a (tidy-up, condition $m > 1$): remove a trailing silent ‘e’, converting relate → relat.
- All three converge on the common stem ‘relat’, which the retrieval index uses as the shared key.

Stemming Steps, Intermediate Forms, and Final Stem

Word	Step Applied	Rule	Intermediate Form	Final Stem
relational	Step 2	ATIONAL → ATE	relate	relat
relation	Step 4	-ION removed (preceded by T, $m > 1$)	relat	relat
relate	Step 5a	Remove trailing silent ‘e’ ($m > 1$)	relat	relat

Conclusion: “relational”, “relation”, and “relate” all reduce to the single Porter stem “relat”. This uniform root ensures that a query on any one form retrieves documents containing all three, satisfying the retrieval module's consistency requirement. Because the algorithm applies conservative, condition-guarded rules rather than a blunt suffix chop, semantically unrelated words that merely share letters (for example, words ending coincidentally in ‘-ate’ with a different meaning) are not incorrectly merged into the same stem class, preserving retrieval precision alongside recall.